

Algorytmy optymalizacji dyskretnej, Lista 3

Błażej Pawluk

9 grudnia 2025

1 Opisy algorytmów

1.1 Klasyczny algorytm Dijkstry

1.1.1 Wyjaśnienie działania algorytmu

Algorytm poczynsz od wierzchołka startowego dokonuje kolejnych iteracji, podczas których dokonuje relaksacji krawędzi do sąsiadów, czyli sprawdza wszystkich sąsiadów wierzchołka. Podczas takiego sprawdzania porównywana jest obecnie najkrótsza trasa do wierzchołka będącego sąsiadem (początkowo ∞ dla wszystkich wierzchołków, 0 dla startowego) z trasą do sprawdzanego wierzchołka + waga krawędzi. Następnie taki wierzchołek z kosztem dodawany jest do zbioru sprawdzonych wierzchołków, a wierzchołek, którego sąsiedzi byli sprawdzani jest odkładany do zbioru wierzchołków, dla których znamy już najkrótszą trasę. Kolejne wierzchołki do sprawdzania wybierane są ze zbioru sprawdzonych w kolejności od najmniejszej trasy.

1.1.2 Pseudokod

Algorithm 1: function `dijkstra(Graph G, Vertex s)`

```
1 dist = [ $\infty$  for each vertex]
2 dist[s] = 0

3 pq = priority queue with (cost, vertex) sorted descending by cost
4 pq.push(0, s)

5 while not pq.empty() do
6   (d, v) = pq.top()
7   pq.pop()
8   if d > dist[v] then
9     continue
10  end
11  for each edge = (u, cost) from G.adj[v] do
12    if d + cost < dist[u] then
13      dist[u] = d + cost
14      pq.push(dist[u], u)
15    end
16  end
17 end

18 return dist
```

1.1.3 Implementacja

Graf przechowywany w strukturze przechowującej n - liczbę wierzchołków, m - liczbę krawędzi, $minCost, maxCost$ - minimalny i maksymalny koszt krawędzi oraz adj - listę sąsiedztwa (`vector<vector<pair<int, long long>>>`). $dist$ jest wektorem o długości n przechowujący wartości `long long`. Jako kolejkę priorytetową używana jest wbudowana struktura `priority_queue` z biblioteki `queue`.

1.1.4 Złożoność obliczeniowa

Złożoności operacji na wbudowanej `priority_queue`:

- tworzenie kolejki priorytetowej = $O(1)$
- `pq.push()` = $O(\log(h))$
- `pq.pop()` = $O(\log(h))$
- `pq.top()` = $O(1)$
- `pq.empty()` = $O(1)$

Kolejka priorytetowa może osiągnąć maksymalny rozmiar $O(m)$ (np. źródło ma krawędź do każdego wierzchołka). W pętli każdy dodany do kolejki element musi być z niej zdjęty operacją `pq.pop()` w złożoności $O(\log m)$. Krawędzi przeglądanych w pętli `for` będzie maksymalnie $O(m)$ (podczas całego algorytmu, nie jednej iteracji przez wszystkie iteracje). Podczas każdej relaksacji w tej pętli wykonywana może być operacja `pq.push()` o złożoności $O(\log m)$. Podsumowując te punkty otrzymujemy $O(m) \times O(\log m) = O(m \times \log m)$

1.2 Algorytm Diała

1.2.1 Wyjaśnienie działania algorytmu

W tej odmianie algorytmu Dijkstry stosujemy kubelki. Każdy z kubelków oznacza odległość od źródła. W tym wariantcie kubelków jest $C+1$, gdzie C to długość najdłuższej krawędzi, zatem kubek k może mieć w sobie krawędzie o odległości $k, k+C+1, k+2C+2, \dots$. Możemy w ten sposób zmniejszyć ilość kubelków, ponieważ wykonując operację na wierzchołku przechowywanym w k -tym kubku możemy otrzymać co najwyżej nowy wierzchołek o odległości $k+C$, więc wykonując to działaniem modulo $C+1$ wierzchołek ten trafi do kubka $k-1$. Początkowo w strukturze kubelków znajduje się jedynie źródło w kubku 0. Wyciągamy go z kubka, przeglądamy dodając ich do odpowiednich kubelków, po czym przechodzimy do innych wierzchołków z kubka, a gdy jest on pusty do kolejnego kubka (gdy będziemy w kubku o indeksie C , to następny jest kubek o indeksie 0).

1.2.2 Pseudokod

Algorithm 2: function dial(Graph G, Vertex s)

```

1  dist = [ $\infty$  for each vertex]
2  dist[s] = 0
3  buckets = array size  $C + 1$  of empty double linked list
4  putInBucket(s, 0)
5   $i = 0$ 
6  while there are any elements in any bucket do
7      while buckets[ $i$ ].empty() do
8           $i = (i + 1) \bmod (C + 1)$ 
9      end
10      $v$  = first element from buckets[ $i$ ]
11     removeFromBucket( $v$ )
12     for each edge = ( $u$ , cost) from  $G.adj[v]$  do
13         if  $d + cost < dist[u]$  then
14              $dist[u] = d + cost$ 
15             putInBucket( $u$ ,  $dist[u]$ )
16         end
17     end
18 end
19 return dist

```

1.2.3 Implementacja

Struktura *buckets*:

- **bucketTop** - wektor o rozmiarze $C+1$, na pozycji k przechowuje wierzchołek będący pierwszym kubelku o indeksie k , lub -1, gdy kubek jest pusty
- **nextInBucket** - wektor o rozmiarze n , na pozycji v przechowuje wierzchołek będący następnikiem wierzchołka v w kubelku, lub -1, gdy nie ma takiego elementu
- **prevInBucket** - wektor o rozmiarze n , na pozycji v przechowuje wierzchołek będący poprzednikiem wierzchołka v w kubelku, lub -1, gdy nie ma takiego elementu
- **bucketOf** - wektor o rozmiarze n , na pozycji v przechowuje informację, w którym kubelku znajduje się obecnie wierzchołek v
- **elements** - licznik elementów dodanych do kubelków
- **removeFromBucket**(v) - usuwa wierzchołek v z kubelka. Działa w złożoności $O(1)$, ponieważ znamy kubek, w którym znajduje się wierzchołek v (**bucketOf**), przestawiamy wskaźniki na poprzedników, następników oraz pierwszy element w kubelku w stałej liczbie operacji, po czym zmniejszamy liczbę **elements**
- **putInBucket**(v , d) - dodaje wierzchołek v do kubelka o indeksie $d \bmod (C+1)$. Działa w złożoności $O(1)$, ponieważ znamy kubek, do którego będzie dodany wierzchołek v , przestawiamy wskaźniki na poprzedników, następników oraz pierwszy element w kubelku w stałej liczbie operacji, po czym zwiększamy liczbę **elements**
- **buckets**[k].**empty**() - zwraca informację, czy kubek o indeksie k jest pusty. Działa w złożoności $O(1)$, ponieważ sprawdza jedynie wartość **bucketTop**[k]

1.2.4 Złożoność obliczeniowa

Wierzchołki są dodawane do kubelków co najwyżej $O(m)$ razy, bo są dodawane maksymalnie raz dla każdej relaksowanej krawędzi, których jest maksymalnie $O(m)$. Każdy dodany element musi być również usunięty, więc operacja `removeFromBucket(v)` jest wykonywana $O(m)$ razy ($O(n)$ dla każdego wierzchołka, ale dodatkowo możemy dodać kilkakrotnie ten sam wierzchołek przy relaksacjach, który jest usuwany w kubelków przed aktualizacją kubelka). Znajdowanie pustego kubelka wymaga maksymalnie $O(C)$ operacji, a pusty kubelek szukany $O(n)$ razy (raz dla każdego wierzchołka). Podsumowując dodawanie do kubelków jest wykonywane $O(m)$ razy, usuwanie $O(m)$ razy i znajdowanie kubelka $O(Cn)$ razy, zatem złożoność algorytmu to $O(m + Cn)$. Problemem jest jednak złożoność pamięciowa, która zależy od wartości C i dla krawędzi o dużych kosztach złożoność szybko rośnie.

1.3 Algorytm RadixHeap

1.3.1 Wyjaśnienie działania algorytmu

W algorytmie Diala korzystaliśmy z kubelków o szerokości (dopuszczalnym zakresie odległości od źródła) 1. W algorytmie RadixHeap szerokości kubelków są zmienne, dla indeksu $0 = 1, 1 = 1, 2 = 2, 3 = 4, \dots, k = 2^{k-1}$, zatem takich kubelków będzie $\log(Cn)$ (bo maksymalna teoretyczna ścieżka między wierzchołkami to nC). Odległość między granicami przedziału jest stała, ale granice są wyznaczane w trakcie programu. Wartość a (początek przedziału w kubelku o indeksie 0) jest równa najkrótszej trasie ostatnio sprawdzonego wierzchołka. Chcąc wybrać nowy wierzchołek do sprawdzenia szukamy pierwszy niepusty kubelek. Jeżeli ma on szerokość 1 (indeks 0 lub 1), to nie ma problemu z wyborem, bo wszystkie wierzchołki w tym kubelku mają taką samą odległość i dokonujemy relaksacji dla ostatniego wierzchołka w kubelku. Natomiast gdy wybrany kubelek ma szerokość ≥ 2 , to zmieniamy wartość a na najkrótszą dotychczasową trasę przechowywaną w tym kubelku, a następnie odpowiednio przenosimy wartości z tego kubelka do wcześniejszych kubelków zgodnie z szerokościami i nową wartością a .

1.3.2 Pseudokod

Algorithm 3: function radixHeap(Graph G, Vertex s)

```
1  dist = [ $\infty$  for each vertex]
2  dist[s] = 0
3  buckets = array of  $\log(Cn)$  empty arrays of pairs (cost, vertex)
4  buckets[0].pushBack(0, s)
5  a = 0
6  while true do
7      cur = 0
8      while cur <  $\log(Cn)$  and buckets[cur].empty() do
9          cur = cur + 1
10     end
11     if cur =  $\log(Cn)$  then
12         break
13     end
14     if cur = 0 or cur = 1 then
15         (d, v) = buckets[cur].back()
16         buckets[cur].popBack()
17         if d > dist[v] then
18             continue
19         end
20         for each edge = (u, cost) from G.adj[v] do
21             if d + cost < dist[u] then
22                 dist[u] = d + cost
23                 putInCorrectBucket(u, dist[u], a)
24             end
25         end
26     else
27         a = findMinValueInBucket(cur)
28         for each (d, v) in bucket[cur] do
29             if d > dist[v] then
30                 continue
31             end
32             putInCorrectBucket(v, dist[v], a)
33         end
34         bucket[cur].clear()
35     end
36 end
37 return dist
```

1.3.3 Implementacja

Zamiast $\log(Cn)$ używamy liczby 65 - w C++ `long long` ma 64 bity, dodajemy kubełek o indeksie 0. Jako kubelki korzystamy ze struktury `vector<vector<pair<long long, int>>>`. Funkcja **putInCorrectBucket**(*v*, *d*, *a*) dodaje wierzchołek do odpowiedniego

wierzchołka poprzez wykonanie bitowej operacji xor na d i a i znalezieniu najbardziej znaczącego bitu, które $+1$ będzie odpowiednim kubelkiem (złożoność $O(1)$, bo maks. 65 operacji przesunięcia bitowego). Funkcja ***findMinValueInBucket***(*cur*) znajduje najmniejszą wartość w kubelku w pesymistycznej złożoności $O(n)$, bo w jednym kubelku może być maks. $O(n)$ wierzchołków.

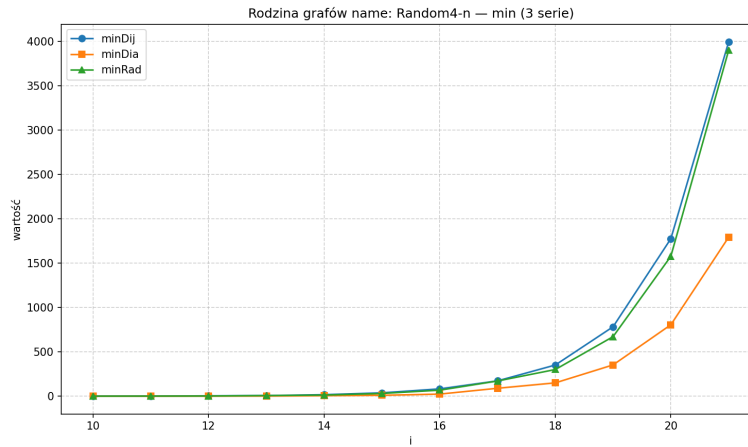
1.3.4 Złożoność obliczeniowa

Szukanie niepustego kubelka najniższego stopnia $O(1)$, przez stałą liczbę (65) kubelków. Wykonywane dla wszystkich wierzchołków $+ \ll n$ razy, gdy trzeba dokonać przestawień między kubelkami, więc $O(n)$. Relaksacje krawędzi $O(m)$ - jak w poprzednich algorytmach, dodanie do kubelka w czasie $O(1)$, więc dla wszystkich krawędzi $O(m)$. W kubelku może się jednocześnie znajdować maksymalnie $O(n)$ wierzchołków, więc znalezienie w nim minimalnej wartości w złożoności $O(n)$, podobnie jak przenoszenie tych wierzchołków do nowych kubelków i czyszczenie kubelka. Podsumowując złożoność całego algorytmu to **$O(n+m)$** .

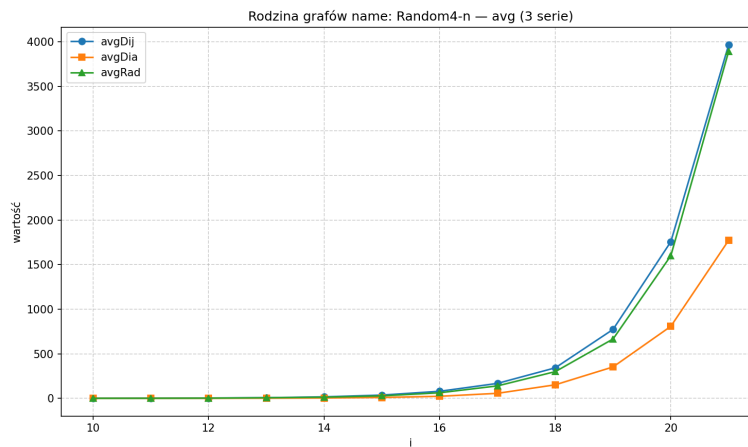
2 Wyniki testów

2.1 Random Graphs: Random4-n, Random4-C

2.1.1 rodzina Random4-n

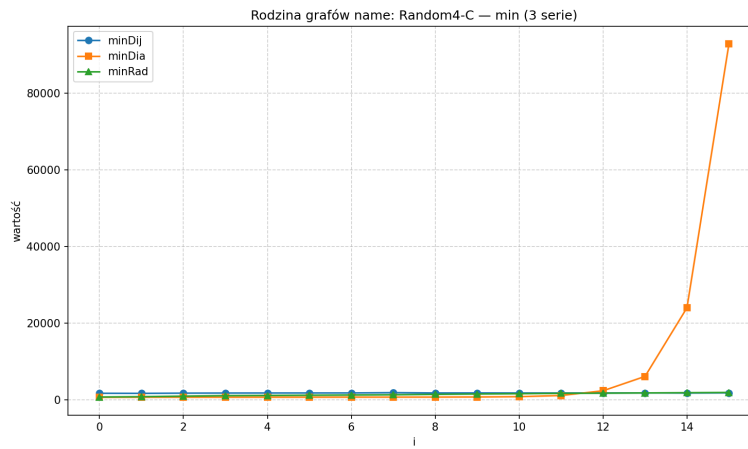


Rysunek 1: Porównanie czasów działania algorytmów Dijkstry, Diala i RadixHeapa - szukanie od wierzchołka o najmniejszym indeksie do wszystkich wierzchołków

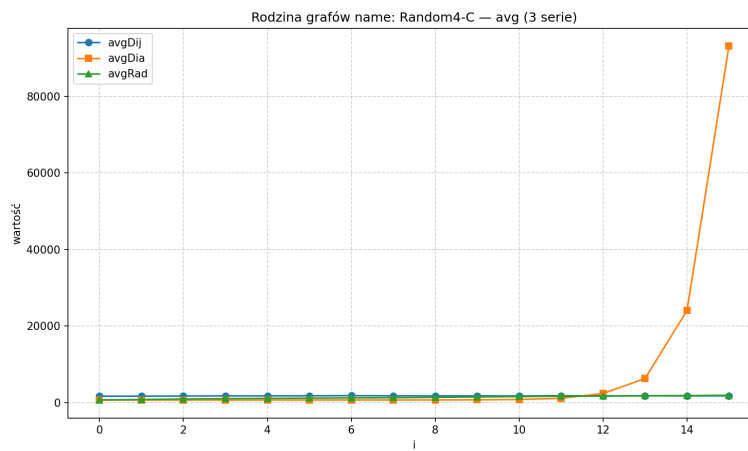


Rysunek 2: Porównanie średnich czasów działania algorytmów Dijkstry, Diala i RadixHeapa - szukanie od losowego wierzchołka do wszystkich wierzchołków (średnia dla 5 iteracji)

2.1.2 rodzina Random4-C



Rysunek 3: Porównanie czasów działania algorytmów Dijkstry, Diala i RadixHeap - szukanie od wierzchołka o najmniejszym indeksie do wszystkich wierzchołków



Rysunek 4: Porównanie średnich czasów działania algorytmów Dijkstry, Diala i RadixHeap - szukanie od losowego wierzchołka do wszystkich wierzchołków (średnia dla 5 iteracji)

2.1.3 Wyniki testów p2p

rodzina	para	Dijkstra	Dial	RadixHeap
Random4-n	1 - 2097152	9051281	9051281	9051281
	1118910 - 310685	10447807	10447807	10447807
	1230060 - 35271	9510613	9510613	9510613
	1387195 - 1411879	9818001	9818001	9818001
	202866 - 1993049	7758793	7758793	7758793
Random4-C	1 - 1048576	3471241820	3471241820	3471241820
	69651 - 1041844	3612704709	3612704709	3612704709
	674419 - 490009	2881715863	2881715863	2881715863
	682105 - 704022	4622473516	4622473516	4622473516
	989279 - 889658	4809709797	4809709797	4809709797

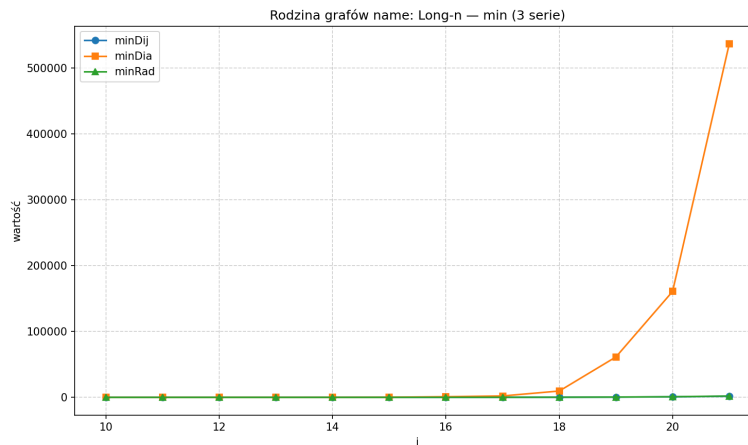
Tabela 1: Porównanie wyników testów p2p

2.1.4 Wnioski

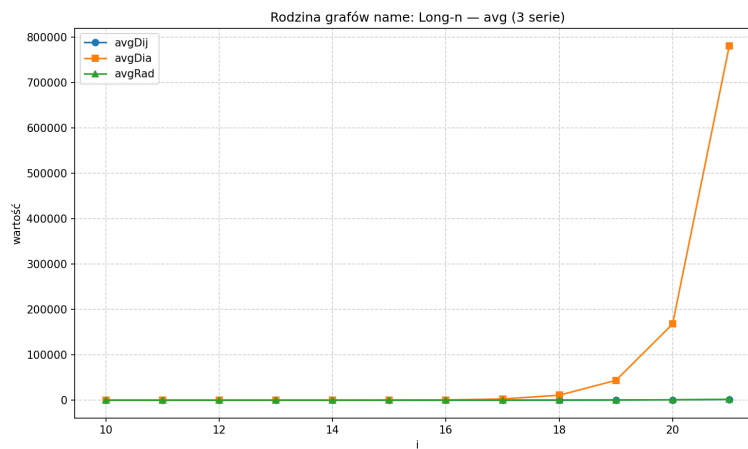
Dla grafów losowych, gdzie wartość C jest proporcjonalna do n (rodzina Random4-n) najlepsze wyniki osiąga algorytm Diała, natomiast gdy wartość C jest o wiele większa od n , to czas wykonywania algorytmu Diała gwałtownie rośnie, co jest spowodowane koniecznością przechowywania ogromnej struktury ($C+1$ list). Wartość C nie ma wpływu na czasy działania algorytmów Dijkstry i RadixHeap. Algorytm Radix Heap wypada minimalnie lepiej od algorytmu Dijkstry, co jest oczekiwane patrząc na złożoność obliczeniową tych algorytmów. Natomiast złożoność obliczeniowa algorytmów Diała i RadixHeap jest odwrotna do otrzymanych wyników, co jest spowodowane wyższą stałą operacji przestawiania elementów do poprzednich kubelków w algorytmie RadixHeap. Dane z tabel pokazują, że wszystkie algorytmy dają takie same wyniki i działają poprawnie.

2.2 Grid Graphs: Long-n, Square-n

2.2.1 rodzina Long-n

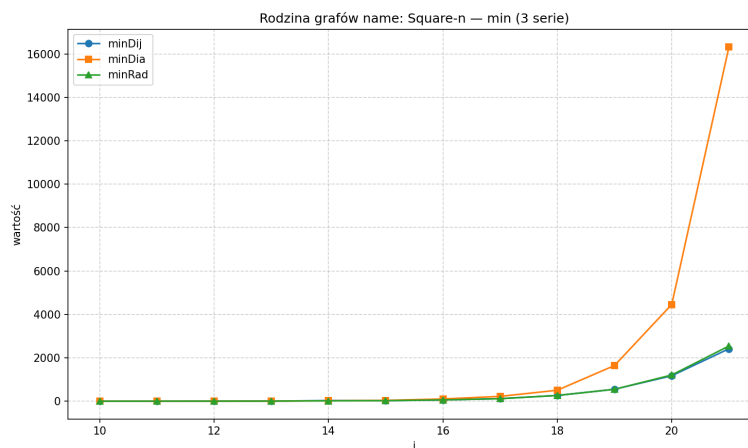


Rysunek 5: Porównanie czasów działania algorytmów Dijkstry, Diała i RadixHeap - szukanie od wierzchołka o najmniejszym indeksie do wszystkich wierzchołków

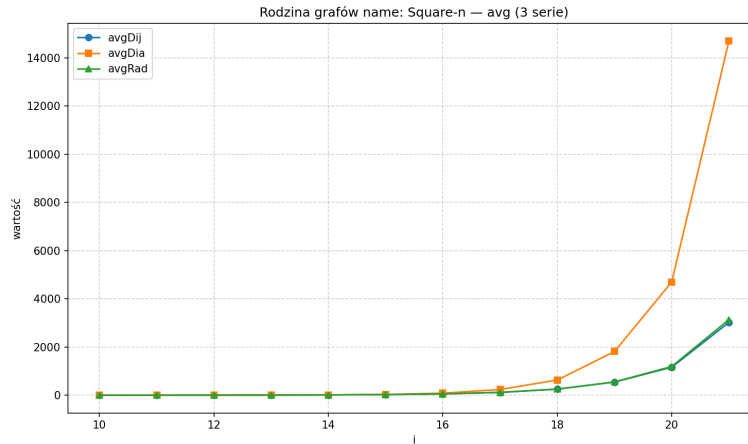


Rysunek 6: Porównanie średnich czasów działania algorytmów Dijkstry, Diala i RadixHeap - szukanie od losowego wierzchołka do wszystkich wierzchołków (średnia dla 5 iteracji)

2.2.2 rodzina Square-n



Rysunek 7: Porównanie czasów działania algorytmów Dijkstry, Diala i RadixHeap - szukanie od wierzchołka o najmniejszym indeksie do wszystkich wierzchołków



Rysunek 8: Porównanie średnich czasów działania algorytmów Dijkstry, Diala i RadixHeap - szukanie od losowego wierzchołka do wszystkich wierzchołków (średnia dla 5 iteracji)

2.2.3 Wyniki testów p2p

rodzina	para	Dijkstra	Dial	RadixHeap
Long-n	1 - 2097152	31336751771	31336751771	31336751771
	35729 - 735967	36048563821	36048563821	36048563821
	564002 - 901499	21765638286	21765638286	21765638286
	1332542 - 153160	41016018643	41016018643	41016018643
	2056575 - 1563311	3225613944	3225613944	3225613944
Square-n	1 - 2096704	714640488	714640488	714640488
	343572 - 1175126	952700567	952700567	952700567
	480812 - 1864599	440024946	440024946	440024946
	740523 - 273216	977733617	977733617	977733617
	1711349 - 1767898	189275343	189275343	189275343

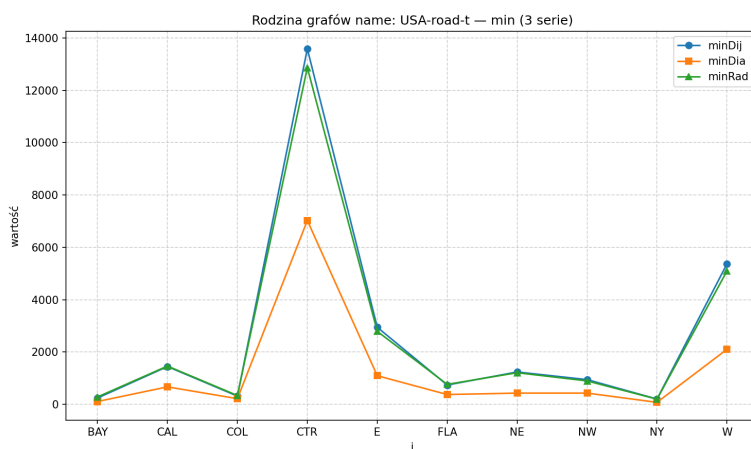
Tabela 2: Porównanie wyników testów p2p

2.2.4 Wnioski

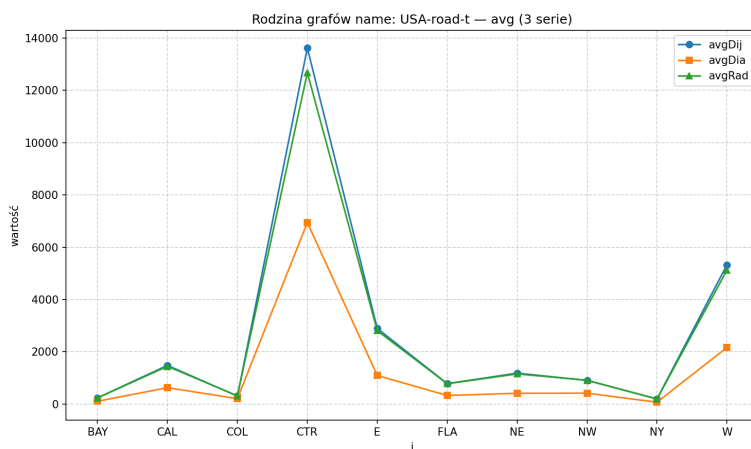
Dla tych rodzin grafów czas wykonywania algorytmu Diala rośnie gwałtownie wraz ze wzrostem wartości n oraz C , o wiele szybciej niż algorytmy Dijkstry oraz RadixHeap. Jest to spowodowane tym, że algorytm Diala ma złożoność zależną od iloczynu C i n , a pozostałe 2 algorytmy tylko od m , które dla tych rodzin jest stosunkowo małe. Dane z tabel pokazują, że wszystkie algorytmy dają takie same wyniki i działają poprawnie.

2.3 USA-road-t

2.3.1 rodzina USA-road-t



Rysunek 9: Porównanie czasów działania algorytmów Dijkstry, Diala i RadixHeap - szukanie od wierzchołka o najmniejszym indeksie do wszystkich wierzchołków



Rysunek 10: Porównanie średnich czasów działania algorytmów Dijkstry, Diala i RadixHeap - szukanie od losowego wierzchołka do wszystkich wierzchołków (średnia dla 5 iteracji)

2.3.2 Wyniki testów p2p

rodzina	para	Dijkstra	Dial	RadixHeap
USA-road-t	1 - 6262104	16307906	16307906	16307906
	1153505 - 3532670	6695864	6695864	6695864
	4080282 - 6023554	36664046	36664046	36664046
	4324713 - 3851041	20885300	20885300	20885300
	5232734 - 1688408	21079702	21079702	21079702

Tabela 3: Porównanie wyników testów p2p

2.3.3 Wnioski

Dla tej rodziny grafów algorytm Diała okazuje się mieć najlepsze wyniki. Jest to spowodowane, tym, że dla grafów odzwierciedlające realne struktury, maksymalne wagi krawędzi nie osiągają takich dużych wartości ($C \cdot n$), co jest podobne do grafów losowych. Dane z tabel pokazują, że wszystkie algorytmy dają takie same wyniki i działają poprawnie.

3 Podsumowanie

Podsumowując algorytm Dijkstry działa w stabilnym czasie dla wszystkich grafów. Jego złożoność zależy od ilości krawędzi. Algorytm Diała w pesymistycznym przypadku (gdy wagi krawędzi są bardzo duże) działa bardzo wolno z powodu bardzo dużej złożoności pamięciowej i złożoności obliczeniowej zależnej od iloczynu maksymalnej wagi i ilości wierzchołków. Natomiast algorytm RadixHeap jest połączeniem tych algorytmów i łączy ich dobre cechy. Jeżeli chcemy optymalne rozwiązania dla wszystkich grafów najlepiej użyć algorytmu RadixHeap, jednak dla przeciętnych przypadków (np. realnych struktur) algorytm Diała, będzie działał wydajniej, ale trzeba wtedy pamiętać o pesymistycznym przypadku.